Problem #1: Design a Python class called Student to track a student's academic performance. Requirements: Constructor (*init*)
Accepts two parameters: name (string) and student_id (string or int). Initializes an empty list scores to store the student's marks. Instance Variables: name → student's name student_id → student's unique ID scores → list of numeric scores (initially empty) Methods: add_score(score) → Appends a new score (int/float) to the scores list. calculate_average() → Returns the average of all scores. If no scores exist, return 0. get_final_grade() → Uses the average to determine result: "Pass" if average ≥ 60 "Fail" otherwise

Test Cases

1. Object Creation student1 = Student("Alice", 101) print(student1.name) # Expected: Alice print(student1.student_id) # Expected: 101 print(student1.scores) # Expected: []
2. Adding Scores

student1.add_score(75) student1.add_score(85) student1.add_score(55) print(student1.scores) # Expected: [75, 85, 55] 3. Calculate Average print(student1.calculate_average())

Expected: (75 + 85 + 55) / 3 = 71.67

4. Final Grade (Pass Case) print(student1.get_final_grade())

Expected: Pass (average = 71.67 ≥ 60)

5. Final Grade (Fail Case) student2 = Student("Bob", 102) student2.add_score(40) student2.add_score(50) print(student2.calculate_average()) # Expected: 45.0 print(student2.get_final_grade()) # Expected: Fail
6. Empty Scores Case student3 = Student("Charlie", 103) print(student3.calculate_average()) # Expected: 0 print(student3.get_final_grade()) # Expected: Fail

```
class Student:
    # Constructor for creating a student object
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.scores = []

    # method for new score
    def add_score(self, score):
        self.scores.append(score)

    # method for finfing average score
    def calculate_average(self):
        if len(self.scores) == 0:
            return 0
        return sum(self.scores) / len(self.scores)
```

```
    # Checking student has passed/failed
    def get_final_grade(self):
        average = self.calculate_average()
        if average >= 60:
            return "Pass"
        else:
            return "Fail"
```

```
# Test Case 1
student1 = Student("Alice", 101)
print(student1.name)
print(student1.student_id)
print(student1.scores)
```

```
Alice
101
[]
```

```
# Test Case 2
student1.add_score(75)
student1.add_score(85)
student1.add_score(55)
print(student1.scores)
```

```
[75, 85, 55]
```

```
# Test Case 3
print(student1.calculate_average())
```

```
71.66666666666667
```

```
# Test Case 4
print(student1.get_final_grade())
```

```
Pass
```

```
# Test Case 5
student2 = Student("Bob", 102)
student2.add_score(40)
student2.add_score(50)
print(student2.calculate_average())
print(student2.get_final_grade())
```

```
45.0
Fail
```

```
# Test Case 6
student3 = Student("Charlie", 103)
print(student3.calculate_average())
print(student3.get_final_grade())
```

```
0
Fail
```

Problem #2: Design a Python class called Gene to represent a DNA sequence. The class should: Store a gene's ID and sequence. Provide methods to: Print the gene's ID and sequence. Count the occurrence of a specific base (A, T, G, C). Calculate the GC content (fraction of G and C bases). Get and set the sequence, ensuring that RNA bases (U) are not allowed.

Test Cases

1. Object Creation gene1 = Gene(101, "ATGCGT")

Expected output (from constructor prints):

I'm a new Gene object!

My constructor got a param: 101

Assign that param to my id instance variable...

Similarly, assigning to my sequence instance variable...

2. Print ID and Sequence gene1.print_id()

Expected: My id is: 101

gene1.print_sequence()

Expected: My sequence is: ATGCGT

3. Base Composition print(gene1.base_composition("A"))

Expected: 1

print(gene1.base_composition("G"))

Expected: 2

print(gene1.base_composition("T"))

Expected: 1

4. GC Content print(gene1.gc_content())

Sequence: ATGCGT ⟶ G=2, C=1 ⟶ GC count=3

Length=6 ⟶ Expected: 3/6 = 0.5

5. Get Sequence print(gene1.get_seq())

Expected: ATGCGT

6. Set Sequence (Valid DNA) gene1.set_seq("ATATGC") print(gene1.get_seq())

Expected: ATATGC

7. Set Sequence (Invalid RNA) gene1.set_seq("AUGC")

```
class Gene:
    # Constructor for gene object
    def __init__(self, gene_id, sequence):
        print("I'm a new Gene object!")
        print("My constructor got a param:", gene_id)
        print("Assign that param to my id instance variable...")
        print("Similarly, assigning to my sequence instance variable...")

        self.gene_id = gene_id
        self.sequence = sequence

    # Print gene ID
    def print_id(self):
        print("My id is:", self.gene_id)

    #   method for Printing gene sequence
    def print_sequence(self):
        print("My sequence is:", self.sequence)

    # method Count a DNA base
    def base_composition(self, base):
        return self.sequence.count(base)

    # Calculate GC content
    def gc_content(self):
        gc = self.sequence.count("G") + self.sequence.count("C")
        return gc / len(self.sequence)

    # Return the sequence
    def get_seq(self):
        return self.sequence

    # Updating the sequence
```

```
    def set_seq(self, new_sequence):
        assert "U" not in new_sequence, "Sorry, no RNA allowed."
        self.sequence = new_sequence
```

```
# Test Case 1
gene1 = Gene(101, "ATGCGT")
```

```
I'm a new Gene object!
My constructor got a param: 101
Assign that param to my id instance variable...
Similarly, assigning to my sequence instance variable...
```

```
# Test Case 2
gene1.print_id()
gene1.print_sequence()
```

```
My id is: 101
My sequence is: ATGCGT
```

```
# Test Case 3
print(gene1.base_composition("A"))
print(gene1.base_composition("G"))
print(gene1.base_composition("T"))
```

```
1
2
2
```

```
# Test Case 4
print(gene1.gc_content())
```

```
0.5
```

```
# Test Case 5
print(gene1.get_seq())
```

```
ATGCGT
```

```
 #Test Case 6
gene1.set_seq("ATATGC")
print(gene1.get_seq())
```

```
ATATGC
```

```
# Test Case 7
gene1.set_seq("AUGC")
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
/tmp/ipykernel_1456/2654457522.py in <cell line: 0>()
      1 # Test Case 7
----> 2 gene1.set_seq("AUGC")

/tmp/ipykernel_1456/2939193556.py in set_seq(self, new_sequence)
     33     # Updating the sequence
     34     def set_seq(self, new_sequence):
---> 35         assert "U" not in new_sequence, "Sorry, no RNA allowed."
     36         self.sequence = new_sequence
     37

AssertionError: Sorry, no RNA allowed.
```

Next steps: Explain error

Start coding or generate with AI.

In [1]:
```python
#Task 1
#Execution of threads in sequence
import threading
import time

start=time.perf_counter()
def do_something():

    time.sleep(1)
    print("Done Sleeping....")
t1=threading.Thread(target=do_something)
t1.start()
t1.join()
finish=time.perf_counter()
print(f'Finished in {round(finish-start,2)} second(s)')
```

```
Done Sleeping....
Finished in 1.0 second(s)
```

In [2]:
```python
#Task 2
#Creatinng list threads throughg for loop
import threading
import time

start=time.perf_counter()
def do_something():

    time.sleep(1)
    print("Done Sleeping....")

threads=[]
for _ in range(10):
    t1=threading.Thread(target=do_something)
    t1.start()
    threads.append(t1)
for i in threads:
    i.join()

finish=time.perf_counter()


print(f"Finished in {round(finish-start,2)} second (s)")
```

```
Done Sleeping....Done Sleeping....

Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Finished in 1.0 second (s)
```

In [14]:
```python
#Task 3
#Creating multiple threads using for loops  using Arguments

import threading
```

```
import time

start=time.perf_counter()
def do_something(seconds):
    print(f'sleeping {seconds} second(s)')

    time.sleep(seconds)
    print("Done Sleeping....")

threads=[]
for _ in range(10):
    t1=threading.Thread(target=do_something,args=[2])
    t1.start()
    threads.append(t1)
for i in threads:
    i.join()

finish=time.perf_counter()


print(f'Finished in {round(finish-start,2)} second (s)')
```

```
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Done Sleeping....
Finished in 2.0 second (s)
```

In [2]:
```
#Task 4
# concurrent futures
#futures from normal use of threading

import concurrent.futures
import time

start=time.perf_counter()
def do_something():

    time.sleep(1)
    return 'Done Sleeping....'


with concurrent.futures.ThreadPoolExecutor() as executor:
    f1=executor.submit(do_something)
```

```
    print(f1.result())

finish=time.perf_counter()

print(f'Finished in {round(finish-start,2)} second (s)')
```

```
Done Sleeping....
Finished in 1.0 second (s)
```

In [7]:
```
#Task 5
# Creating Multiple Threads using ThreadPool Executor
#Using For loop

import concurrent.futures
import time

start = time.perf_counter()


def do_something(seconds):
    print(f"Sleeping  {seconds} second(s)")

    time.sleep(seconds)
    return f"Done Sleeping {seconds} second(s)...."


with concurrent.futures.ThreadPoolExecutor() as executor:
    results = [executor.submit(do_something, 1) for _ in range(10)]

    for f in concurrent.futures.as_completed(results):
        print(f.result())

finish = time.perf_counter()

print(f"Finished in {round(finish-start,2)} second (s)")
```

```
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Sleeping  1 second(s)
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Done Sleeping 1 second(s)....
Finished in 1.0 second (s)
```

In [13]:
```python
# Task 6
# Creating Multiple Threads using concurrent Futures
#Using Thread Pool Executor
import concurrent.futures
import time

start = time.perf_counter()


def do_something(seconds):
    print(f"Sleeping  {seconds} second(s)")
    time.sleep(seconds)
    return f"Done Sleeping {seconds} second(s)...."


with concurrent.futures.ThreadPoolExecutor() as executor:
    secs = [5, 4, 3, 2, 1]

    # FIX: Changed 'secs' to 'sec' inside executor.submit()
    results = [executor.submit(do_something, sec) for sec in secs]

    for f in concurrent.futures.as_completed(results):
        print(f.result())

finish = time.perf_counter()
print(f"Finished in {round(finish-start,2)} second (s)")
```

```
Sleeping  5 second(s)
Sleeping  4 second(s)
Sleeping  3 second(s)
Sleeping  2 second(s)
Sleeping  1 second(s)
Done Sleeping 1 second(s)....
Done Sleeping 2 second(s)....
Done Sleeping 3 second(s)....
Done Sleeping 4 second(s)....
Done Sleeping 5 second(s)....
Finished in 5.0 second (s)
```

In [9]:
```python
#Task 7
# Using the built-in map() method
import concurrent.futures
import time

start=time.perf_counter()
def do_something(seconds):
    print(f"Sleeping  {seconds} second(s)")

    time.sleep(seconds)
    return f"Done Sleeping {seconds} second(s)...."


with concurrent.futures.ThreadPoolExecutor() as executor:
    secs=[5,4,3,2,1]
    results=executor.map(do_something,secs)

for result in results:
    print(result)
```

```
finish=time.perf_counter()
print(f"Finished in {round(finish-start,2)} second (s)")
```

```
Sleeping  5 second(s)
Sleeping  4 second(s)
Sleeping  3 second(s)
Sleeping  2 second(s)
Sleeping  1 second(s)
Done Sleeping 5 second(s)....
Done Sleeping 4 second(s)....
Done Sleeping 3 second(s)....
Done Sleeping 2 second(s)....
Done Sleeping 1 second(s)....
Finished in 5.0 second (s)
```

In [5]:

```
#Task 8
#Download Audio Files and Print the Time taken to Download the file

import time
import requests

urls = [
    "https://wikimedia.org",
    "https://wikimedia.org",
    "https://wikimedia.org",
    "https://wikimedia.org",
    "https://wikimedia.org",
    "https://wikimedia.org",
    "https://wikimedia.org",
    "https://wikimedia.org",
]


start=time.perf_counter()

for u in urls:
    audio_bytes=requests.get(u).content
    a_name=u.split('/')[-1]
    with open(a_name,"wb") as a_file:
        a_file.write(audio_bytes)
        print(f'{a_name} was Downloaded....')

finish=time.perf_counter()
print(f'Finished in {round(finish-start,2)} seconds')
```

```
wikimedia.org was Downloaded....
wikimedia.org was Downloaded....
wikimedia.org was Downloaded....
wikimedia.org was Downloaded....
wikimedia.org was Downloaded....
wikimedia.org was Downloaded....
wikimedia.org was Downloaded....
wikimedia.org was Downloaded....
Finished in 1.36 seconds
```

In [ ]:

Multi Threading using Priority Queue

In [1]:
```
#Example 1
import threading
import queue
import time

exitFlag = 0

class myThread(threading.Thread):
    def __init__(self, threadID, name, q):
        threading.Thread.__init__(self)
        self.threadID = threadID
        self.name = name
        self.q = q

    def run(self):
        print("Starting " + self.name)
        process_data(self.name, self.q)
        print("Exiting " + self.name)


def process_data(threadName, q):
    while not exitFlag:
        queueLock.acquire()

        if not workQueue.empty():
            data = q.get()
            queueLock.release()
            print("%s processing %s" % (threadName, data))
        else:
            queueLock.release()
            time.sleep(1)


thread_list = ["Thread-1", "Thread-2", "Thread-3"]

name_list = ["A", "B", "C", "D", "E"]

queueLock = threading.Lock()
workQueue = queue.Queue(10)

threads = []
threadID = 1

# Create new threads
for thread_name in thread_list:
    thread = myThread(threadID, thread_name, workQueue)
    thread.start()
    threads.append(thread)
    threadID += 1

# Fill the queue
queueLock.acquire()

for item in name_list:
    workQueue.put(item)
```

```
queueLock.release()

# Wait for the queue to empty
while not workQueue.empty():
    pass

# Notify threads it's time to exit
exitFlag = 1

# Wait for all threads to complete
for t in threads:
    t.join()

print("Exit Main Thread")
```

```
Starting Thread-1
Starting Thread-2
Starting Thread-3
Thread-1 processing AThread-3 processing B
Thread-3 processing C
Thread-3 processing D
Thread-3 processing E

Exiting Thread-1
Exiting Thread-2
Exiting Thread-3
Exit Main Thread
```

EXAMPLE-SCENARIO: "A food delivery application receives orders from multiple customers at the same time. "Each order needs to be processed independently. Write a simple Python program using "multithreading to process three customer orders simultaneously"

In [2]:
```
#Example 2
import threading
import time

def process_order(customer):
    print(customer, "order received")
    time.sleep(2)  # Simulate order processing time
    print(customer, "order completed")


# Create threads for different customers
t1 = threading.Thread(
    target=process_order,
    args=("Arun",)
)

t2 = threading.Thread(
    target=process_order,
    args=("Priya",)
)
```

```
t3 = threading.Thread(
    target=process_order,
    args=("Kumar",)
)

# Start all threads
t1.start()
t2.start()
t3.start()

# Wait for all threads to complete
t1.join()
t2.join()
t3.join()

print("All orders processed")
```

```
Arun order received
Priya order received
Kumar order received
Arun order completed
Priya order completed
Kumar order completed
All orders processed
```

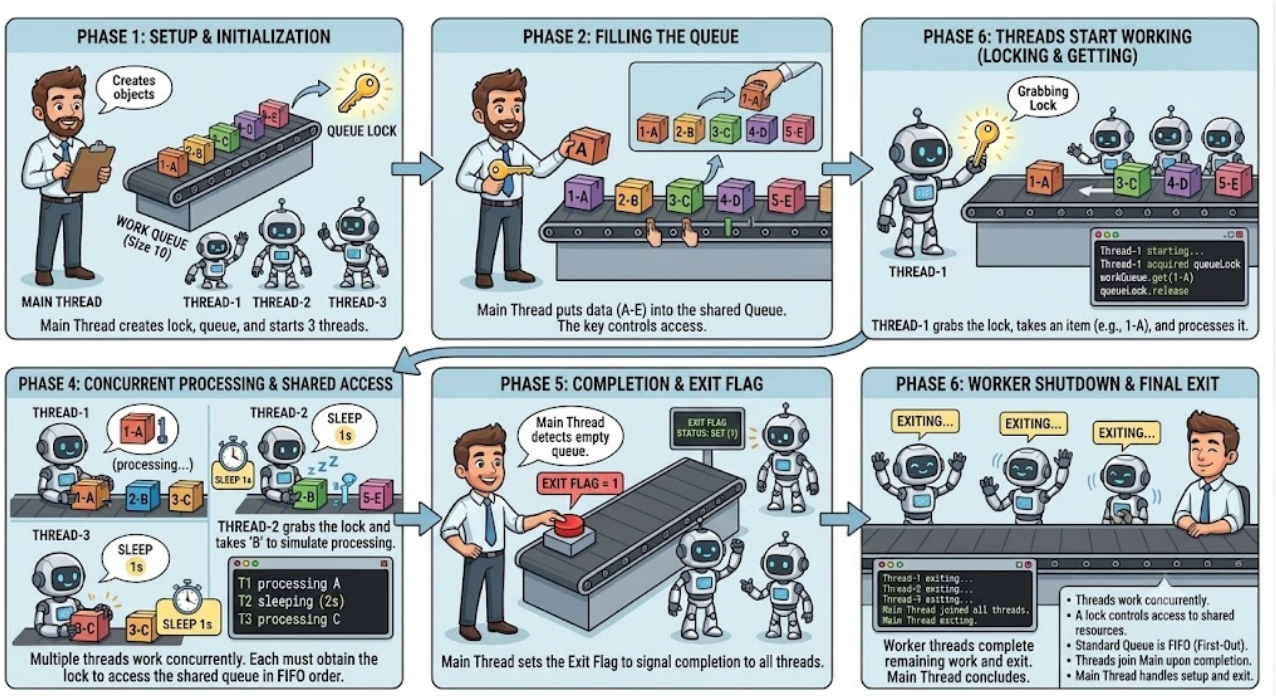
In [ ]:

In [ ]:

In [ ]:

Multi Threading Using prioty Queue

Flow Diagram



Printing Threads

In [1]:
```
import threading
print('Current Executing Thread:', threading.current_thread().getName())
```

```
Current Executing Thread: MainThread
/tmp/ipykernel_35092/448115826.py:2: DeprecationWarning: getName() is depr
ecated, get the name attribute instead
  print('Current Executing Thread:', threading.current_thread().getName())
```

In [2]:
```
from threading import *
def display():
    for i in range(10):
        print('Child Thread')
# Creation of Thread Object to Execute Display
t=Thread(target=display)
t.start()
for i in range(10):
    print('Main Thread')
```

```
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
```

In [3]:
```
from threading import *
def display():
    for i in range(10):
        print('Child Thread')
#Thread Object to Execute Display
t=Thread(target=display)
t.start()
for i in range(10):
    print('Main Thread')
```

```
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
```

In [4]:
```
from threading import *
def display():
    print('Child Thread Name:',current_thread().getName())
    print('')
t=Thread(target=display)

t.start()
print('Main Thread Name:',current_thread().getName())
```

```
Child Thread Name: Thread-7 (display)

Main Thread Name: MainThread
/tmp/ipykernel_35092/2536291584.py:3: DeprecationWarning: getName() is dep
recated, get the name attribute instead
  print('Child Thread Name:',current_thread().getName())
/tmp/ipykernel_35092/2536291584.py:8: DeprecationWarning: getName() is dep
recated, get the name attribute instead
  print('Main Thread Name:',current_thread().getName())
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
```

In [5]:
```
from threading import *
class MyThread(Thread):
    def run(self):
        for i in range(10):
            print('Child Thread')

t=MyThread()
t.start()
for i in range(10): # Both Loops execute concurrently
```

```
    print('Main Thread')
```

```
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
```

In [6]:
```
# example of extending the Thread class
from time import sleep
from threading import Thread

# custom thread class
class CustomThread(Thread):
    # override the run function
    def run(self):
        sleep(1)
        print('This is coming from another thread')

thread = CustomThread()
thread.start()
print('Waiting for the thread to finish')
thread.join()
```

```
Waiting for the thread to finish
This is coming from another thread
```

In [7]:
```
import threading
import time
def print_numbers():
    for i in range(1, 6):
        print(f"Number: {i}")
        time.sleep(1)
def print_letters():
    for letter in ['A', 'B', 'C', 'D', 'E']:
        print(f"Letter: {letter}")
        time.sleep(1)
t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)
t1.start()
t2.start()
t1.join()
t2.join()
print("Both threads completed.")
```

```
Number: 1
Letter: A
Number: 2
Letter: B
Number: 3
Letter: C
Number: 4
Letter: D
Number: 5
Letter: E
Both threads completed.
```

In [ ]: